

JEDCO No-Show App — Comprehensive Security Assessment

Scope: cl-erage/noshow-app @ branch claude/web-app-security-audit-8T7KD (Node/Express backend on Railway + Postgres + Cloudflare R2; React/Vite PWA on Vercel)

Method: Read-only static review, dependency review, attack-surface mapping, threat modeling. No exploitation, no probing, no changes.

Context that matters: Airport-operations system at KAIA (Jeddah). Data includes passenger nationality, pax type (Umrah/Hajj/transit/resident), flight changes, identity-document scans (uploads). Operates under Saudi PDPL. A handful of operators; supervisors hold near-total power.

1. Executive Summary

The application is small, the code is generally clean, and many basics are correct: parameterised SQL, bcrypt (cost 12) for PINs, Helmet, an explicit-origin CORS policy, HMAC-signed session tokens, an `invite_tokens` flow rather than open registration, an `audit_log` table, R2 object storage abstracted from the DB. There is no SQLi, no obvious XSS sink, no SSRF surface, no XXE, no deserialisation gadget chain.

However, the session model has a structural defect with hard production consequences: **tokens are stateless 12-hour HMACs with no server-side revocation, no per-user secret, no rotation on PIN reset, and no active/existence re-check**. Combined with **horizontal IDOR across every report endpoint** (any authenticated staff can read, mutate, attach files to, and re-status any report — including reopening "closed" cases or vandalising another shift's work), and **client-controlled submitted_by / Nusuk-user attribution**, this is enough to enable real abuse by an insider or by anyone who briefly gets hold of a staff device.

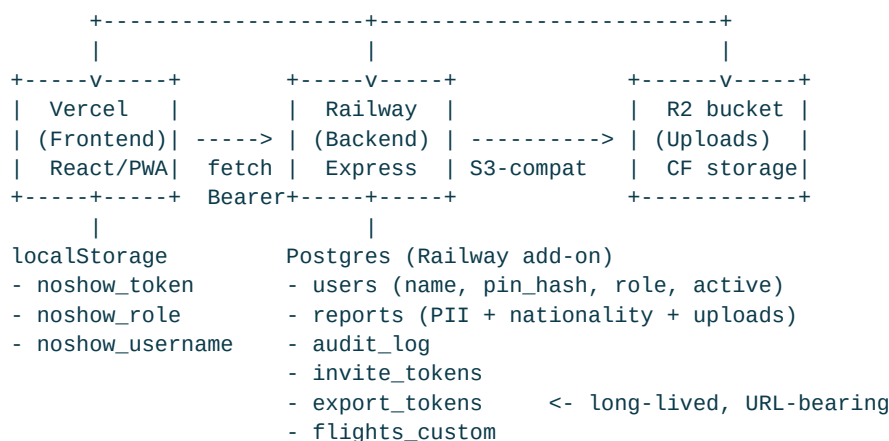
A second class of risk centres on **supervisor power concentration**: a single supervisor can reset any other supervisor's PIN, deactivate any account, mint indefinite-lifetime export tokens that stream the entire reports table and audit log over plain URLs, and (because of the revocation gap) cannot be locked out for up to 24 hours after detection.

Overall maturity: **Level 2 — Repeatable**. Controls exist, but key controls are partial (revocation, ownership, monitoring, alerting, segregation of duties). Recommend addressing the Critical/High items before next deployment cycle. None of the Critical findings require architectural rewrites.

Risk verdict: Acceptable for a small trusted-operator app *today* on the assumption that staff are vetted and devices controlled; **not acceptable** for any scenario involving high turnover, BYOD, lost devices, third-party integrators, or a hostile insider. Most fixes are 1–20 lines.

2. Attack-Surface Map





Service Worker

- share-target POST -> caches files locally -> uploaded via FormData

Authentication surfaces

Surface	Auth	Notes
POST /api/auth/login	none (public)	rate-limited 30/15m per IP; returns HMAC token
POST /api/auth/register	invite token	shares the same rate-limit bucket as login
GET /api/auth/invite/:token	none (public)	leaks role + expiry; token in URL
/api/flights/* /api/reports/* /api/analytics/* /api/users/*	Bearer HMAC	session-style
/api/export/live-state /api/export/audit-log /api/export/ping	?token= (URL) or x-export-token	long-lived, no expiry
/api/files/:filename	Bearer HMAC	no per-report ownership check
/api/users/* /api/export/tokens* /api/flights POST/DELETE	Bearer + role=='supervisor'	sole privileged class

Storage / data exposures

- Tokens (session, invite, export) stored as **plaintext** in Postgres
- Uploaded ID scans persisted in R2; deleted only when report transitions to **closed**
- All cross-shift PII visible to all staff (no per-user/per-shift scoping)

Third-party surfaces

- images.kiwi.com (airline logos) — CSP-allowed in `imgSrc`
- Google Fonts (fonts.googleapis.com, fonts.gstatic.com) — CSP-allowed
- Vercel CDN, Railway proxy, Cloudflare R2

Outbound surfaces: none from user input (no SSRF reachable). Outbound calls are all to fixed AWS SDK + Cloudflare R2 endpoint via env config.

No public endpoints discovered that bypass auth aside from `/api/auth/login`, `/api/auth/register`, `/api/auth/invite/:token`, `/api/health`. No GraphQL, no WebSocket, no debug routes.

3. Critical Findings (act first)

C-1 — Tokens cannot be revoked; deactivated/compromised accounts stay live for up to 24 h

Severity: **Critical** · **CVSS v3.1:** 8.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)

Affected: `backend/middleware/auth.js:3-11`, `backend/routes/auth.js:13, 34-49`

Root cause. `verifyToken` only validates an HMAC over `role:b64name:window` where `window = floor(now/12h)` and accepts the current or previous window. There is **no DB lookup**, so:

- A deactivated user (`users.active = false`) keeps a usable token for up to ~24 h.
- A user with a reset PIN keeps the old session (the PIN hash isn't part of the token).
- A user whose role was demoted keeps the old role until token natural expiry.
- A leaked token cannot be invalidated — there is nothing to invalidate against.
- The supervisor `Deactivate` button in `UsersPage` is therefore *misleading* — it conveys an immediate effect that doesn't happen.

Attack scenario. Staff device left on a counter → opportunist copies `localStorage.noshow_token` from DevTools → uses it from anywhere for the rest of the 12 h window. When a supervisor deactivates the account, the attacker continues operating for another full window.

Remediation.

- Bind tokens to a per-user secret (`users.token_version` integer). Include it in the HMAC input, bump it on PIN reset, deactivation, and a new "force-logout" action. Verify by reading the `users` row each request (cache 30 s if perf matters — this app has <50 users).
- Shorten the window to 1–2 h.
- Add explicit `iat/exp` to the token payload and check them.
- Add a `DELETE /api/auth/logout` and a `DELETE /api/users/:id/sessions` that bumps `token_version`.

Detection. Log every 401 response with the username (if recoverable). Alert on >N 401s for a given username across IPs.

C-2 — Horizontal IDOR across all report endpoints (any staff can read/edit/attach to/reopen any report)

Severity: **Critical** · **CVSS v3.1:** 8.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)

Affected: `backend/routes/reports.js` — `GET /`, `GET /:id`, `PUT /:id`, `PATCH /:id`, `POST /:id/files`, `POST /:id/nusuk`. Only `DELETE /:id` and `DELETE /:id/files/:filename` are supervisor-gated.

Root cause. There is no `submitted_by/owner_id/shift` gate on read or write. `submitted_by` is purely descriptive, and even that is set from `req.body` (see C-3).

Concrete impact.

- Staff A can `PATCH /api/reports/<staff-B-report>` to flip status from `flight_confirmed` → `under_process`, rewriting comments and corrupting handover/shift summaries.
- Staff A can attach attacker-supplied files to staff B's report, polluting evidence chain.
- Staff A can mark `nusuk_received` for someone else's Umrah pax with the supplied `user` field.
- `pax_count` on a closed report can be re-edited to inflate or destroy analytics.

Remediation.

- Add `submitted_by_user_id` (FK to `users.id`) populated from `req.username` server-side.
- Restrict mutation endpoints: `PUT /:id`, `PATCH /:id`, `POST /:id/files`, `POST /:id/nusuk`, `DELETE /:id/files/:filename` → owner or supervisor; reads can remain shared if that's the business rule.
- Specifically forbid transitioning out of `closed` for non-supervisors.
- Validate state transitions: `under_process` → `flight_confirmed` → `closed`; `closed` → `under_process` only for supervisors with an audit reason.

C-3 — Client-supplied attribution (`submitted_by`, Nusuk user) breaks accountability and the audit trail

Severity: High · **CVSS:** 6.5 (AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N)

Affected: `backend/routes/reports.js:341,371` (`submitted_by` from `req.body`), `:647-649` (Nusuk user from `req.body`).

Root cause. Both fields are taken directly from the request body. The `audit_log` captures `req.username` correctly, but report rows and the Nusuk-by field are forgeable. A malicious staff can attribute an action to anyone, including non-existent or supervisor names.

Remediation. Always set `submitted_by = req.username`, `nusuk_by = req.username`. Remove these fields from the API request schema.

C-4 — Single rogue/compromised supervisor takes over the whole tenant

Severity: High · **CVSS:** 7.6 (AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H)

Affected: `backend/routes/users.js:43-60` (`PATCH /:id/pin`), `:25-40` (`PATCH /:id/active`), `:64-85` (`POST /invite`).

Root cause. A supervisor can (a) reset any other supervisor's PIN to a value they know, (b) deactivate every other supervisor account, (c) mint new supervisor invites at will, (d) issue export tokens that exfiltrate the entire reports + audit log via plain URL. No four-eyes/dual-control, no per-action MFA, no notification to the victim.

Remediation.

- Prevent a supervisor from PATCHing their own role/active, and from PIN-resetting another supervisor without a second supervisor's approval (request → confirm flow).
- Email/notify the victim user on PIN reset / deactivation / supervisor-invite creation / export-token creation.
- Consider an immutable break-glass "owner" account whose PIN cannot be reset by other supervisors.

C-5 — Export-token leak via URL (browser history, server logs, referer)

Severity: High · **CVSS:** 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

Affected: `backend/routes/export.js:8-21,99-135`; `AccessManagement.jsx:11-13` constructs the URL-bearing link by design.

Root cause. Tokens flow as `?token=...` query parameter. They are stored plaintext in Postgres (`export_tokens.token`). They have no expiry — only manual `revoked` or `rotate`. Live state and audit log endpoints both accept the URL token, will be cached in browser history / Excel connection strings / CDN edge nodes, and the audit-log endpoint's `LIMIT $1` clamp is `5000` rows by default — i.e. one fetch streams thousands of audit entries.

Remediation.

- Hash tokens in DB (`crypto.timingSafeEqual` on SHA-256 of the presented token).
- Require the header form (`x-export-token`) and reject `?token=`, OR keep query support but explicitly mark these tokens as "read-only, low sensitivity" and warn the supervisor when they create one.
- Add `expires_at` to `export_tokens`, default 90 days, rotate automatically.
- Set `Cache-Control: no-store` and `Referrer-Policy: no-referrer` on every export response.
- Stream the audit log paginated (cursor or `since=`), not `LIMIT 5000`.
- Audit-log export endpoint should *not* be reachable with `role=view` tokens; today the `role` column is created but never enforced.

C-6 — role column on export tokens is never enforced

Severity: High · **CVSS:** 6.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

Affected: `backend/routes/export.js:99,118` — both `live-state` and `audit-log` use `requireToken` with no role gate; any valid token reads both.

Root cause. UI offers View/Analyst/Admin selectors (`AccessManagement.jsx:120-122`) but the server ignores the value.

Remediation. Enforce role: `view` → live-state only, `analyst` → +audit, `admin` → +write (if you ever add write). Add an integration test.

4. High Findings

H-1 — File-download endpoint has no per-report authorization

Affected: `backend/server.js:63-76`. Anyone with any valid Bearer token can GET `/api/files/<filename>` for any upload, given the filename. Filenames are `${Date.now()}-${Math.round(Math.random()*1e9)}.<ext>` — only ~30 bits of entropy from `Math.random()` plus a guessable timestamp; not safely unguessable. They also leak in API responses (`file_paths` returned via GET `/api/reports/:id`).

Impact. Any authenticated user can fetch every uploaded ID document, even from reports tied to other shifts/users.

Remediation. Either (a) join through `reports.file_paths` and require the requesting user own/supervise the parent report, or (b) move to per-report-scoped signed URLs from R2 with short TTL.

H-2 — Username enumeration via login response & timing

Affected: `backend/routes/auth.js:91-96`. Three branches:

- name not found → 401 "Invalid name or PIN."
- name found, not active → 403 "Account is deactivated. Contact your supervisor."
- name found, wrong pin → 401 "Invalid name or PIN."

The deactivated branch leaks existence; the bcrypt branch is observably slower than the not-found branch (~250 ms vs <10 ms).

Remediation. Return identical 401 for all of {no user, inactive, bad pin}. Run a dummy `bcrypt.compare(pin, DUMMY_HASH)` on the no-user path to equalise timing. Surface "deactivated" via the supervisor channel, not the login screen.

H-3 — PIN brute-force ceiling is too generous for 6-digit numeric secrets

Affected: `backend/routes/auth.js:15-21, 57-58`. $\text{PIN} \geq 6$ digits → only 10^6 space (10^7 for 7-digit, etc.). Rate limit is per-IP, 30 attempts / 15 min. Attacker with even 10 IPs gets 300 attempts/15 min $\approx$ 28,800/day → 10^6 space brute-forced in ~35 days against a single known username; with botnets, minutes.

No account lockout after N failed attempts. No CAPTCHA. No alerting on burst failures.

Remediation. Add a per-account failed-attempt counter (`users.failed_attempts`, `users.locked_until`) — lock for exponential backoff after 5/10/20 failures. Keep IP limiter as defence-in-depth. Consider requiring 8+ digit PINs or switching to passphrase, given operators on shared devices.

H-4 — loginLimiter is shared by /login and /register; X-Forwarded-For may be spoofable

Affected: `backend/routes/auth.js:15-21, 74, 107`, `backend/server.js:32` (`trust proxy: 1`).

- Register and login share the same 30/15m bucket → attacker burning register attempts also DoS's legitimate logins from the same IP.

- `app.set('trust proxy', 1)` trusts the *first* proxy hop. If anyone can hit the backend directly (the Railway host is publicly addressable and frequently is — confirm), `X-Forwarded-For` becomes attacker-controlled, defeating the limiter.

Remediation. Create separate limiters. Either restrict trusted proxies explicitly (e.g. `trust proxy: <CIDR of Railway edge>`), or use `req.socket.remoteAddress` for limiter key when `req.ip` derivation is uncertain. Test by hitting the Railway URL directly with a spoofed XFF and confirming the limit applies.

H-5 — No CSP / security headers on the Vercel frontend

Affected: `frontend/vercel.json` — only rewrites; no `headers`: block.

Impact. Tokens live in `localStorage` (`frontend/src/utils/auth.js`), so any XSS = full account takeover (token theft + indefinite use within window, see C-1). The backend Helmet CSP only covers backend-served pages (which is mostly just `/user-manual` and JSON APIs); the SPA itself ships with **zero CSP, no Strict-Transport-Security, no Referrer-Policy, no X-Content-Type-Options** — Vercel sends some defaults but you're not pinning them.

Remediation. Add `headers` to `vercel.json` for:

- `Content-Security-Policy` (default-src 'self'; connect-src 'self' <RAILWAY_API>; img-src 'self' data: https://images.kiwi.com; style-src 'self' 'unsafe-inline'; object-src 'none'; frame-ancestors 'none')
- `Strict-Transport-Security`: `max-age=63072000; includeSubDomains; preload`
- `Referrer-Policy`: `no-referrer`
- `X-Content-Type-Options`: `nosniff`
- `Permissions-Policy` to disable geolocation/microphone/camera (PWA share API needs none of these).

H-6 — Auth tokens stored in `localStorage` (XSS = total takeover)

Affected: `frontend/src/utils/auth.js:1-15`. Combined with H-5 there's no defence-in-depth. Even without an XSS today, any future supply-chain compromise (one of `react-router-dom`, `recharts`, `exceljs`, `jspdf`, `browser-image-compression`, `vite-plugin-pwa`) injecting a script reads the token straight from `localStorage`.

Remediation. Move auth to `HttpOnly; Secure; SameSite=Strict` cookie. Backend would then need a CSRF token for state-changing endpoints (double-submit cookie works; you already have `cookie-parser` in dependencies but unused). This is a meaningful refactor; deprioritise vs C-1 unless you adopt cookies anyway.

H-7 — `validatePin` requires `^\d{6,}$` but UI caps at 12 — server `maxLength` not enforced

Affected: `backend/routes/auth.js:57-71` accepts arbitrarily long digit strings. Combined with `bcrypt.hash(pin, 12)`, an attacker submitting a 1-million-digit "PIN" via `/register` (within the 100 KB body limit, ~100 K digits) causes a multi-second `bcrypt` CPU pin. Repeated, this is a CPU DoS.

Remediation. Add `pin.length <= 12` to `validatePin`. Also cap login `pin` length pre-`bcrypt`.

H-8 — Multipart upload: trusts client MIME, no magic-byte verification, polyglots possible

Affected: `backend/routes/reports.js:60-78`. `multer` `fileFilter` uses `file.mimetype` (client-supplied) only. The `streamFile` path (`backend/storage.js:34-43`) serves the file back with the *stored* `Content-Type`, inline by default. An attacker who can upload (any staff) could:

- Upload a PDF-with-JS payload (`application/pdf` accepted) — browser PDF viewer may execute JS within sandbox, but JS-in-PDF is a known phishing/credential-exfil surface.
- Upload an HTML polyglot mislabelled as `image/jpeg`. Most browsers won't render HTML if the response says `image/jpeg`, but mobile webviews and older browsers vary.

Remediation.

- Use a magic-byte sniff (e.g. `file-type` npm package) and reject mismatches.
- Always serve with `Content-Disposition: attachment` for non-image types; or always `attachment`, given users have a download flow.
- Add `X-Content-Type-Options: nosniff` to file responses (Helmet sets it on `/api/*` but `/api/files/:filename` uses `res.send/sendFile` which should still inherit).
- Add Content-Disposition filename sanitisation (currently `encodeURIComponent(filename)` is OK but a bare basename would be safer).

H-9 — Audit log incomplete: no auth events, no rate-limit hits, no failures, no IPs

Affected: `audit_log` table; only mutation actions are logged. **Not** logged: logins (success/fail), failed PIN attempts, rate-limit hits, role changes, deactivations, export-token mints/revocations/rotations/usage, file downloads, file upload virus/MIME rejections, IP addresses, user-agents.

Impact. Even after a breach you cannot answer "who logged in from where, when?", "did anyone scrape `/api/reports?`", "was the export token used outside business hours?".

Remediation. Add columns `actor_ip`, `actor_ua` to `audit_log`. Add new actions: `login_success`, `login_failure`, `rate_limited`, `export_token_used`, `pin_reset_by`, `deactivated`, `invite_created`, `file_downloaded`. Ship Postgres backups + Railway logs to an external sink (Logflare, Datadog, BetterStack).

5. Medium Findings

M-1 — `invite/:token` validator leaks invite metadata to anyone with the token (no rate limit)

`GET /api/auth/invite/:token` is unauthenticated and unthrottled (route is not under `loginLimiter`). It returns `role` + `expiresAt`, lets an attacker who finds a leaked link (Slack, email, SMS) recon the role to claim before the legitimate operator. Add the limiter, return a constant response shape, and consider single-use → claim-by-confirmation.

M-2 — Tokens (session/invite/export) stored as plaintext in DB

A Postgres dump or read-only DB compromise immediately yields all live sessions, all unused invites, and all export tokens. Store SHA-256 hashes (export and invite tokens — they're shown to user once anyway). Session tokens are HMAC-derived and don't have a server-side row.

M-3 — `requireAuth` uses non-constant-time HMAC comparison

`backend/routes/auth.js:43` does `sig === expected`. JavaScript string `===` is short-circuit and could leak timing on long HMACs across slow networks. Use `crypto.timingSafeEqual(Buffer.from(sig, 'hex'), Buffer.from(expected, 'hex'))` after length check. (Risk small but free to fix.)

M-4 — `setInterval(autoCloseReports, 60s)` with no overlap guard

If one run exceeds 60 s (e.g. R2 deletions for a closed report stalling), the next run starts before it finishes. With `UPDATES` these are idempotent but you can double-emit `auto_close` audit entries and double-fire `deleteFile`. Add a simple `isRunning` flag.

M-5 — No segregation of read vs write privileges

Every authenticated user (staff) can `POST /api/reports` and `PATCH/PUT` any. There is no read-only role. In an airport setting, "auditor", "external observer", "trainee" are common roles. Today an auditor must use an export token (URL-leak risk per C-5) or a full staff account.

M-6 — Unlimited body-size / payload growth on `/api/reports` & `/api/reports/:id/files`

20 MB × 10 files = 200 MB per request, no daily quota, no per-user storage cap. A malicious staff could exhaust R2 storage (cost), the audit log (insert pressure), and the `file_paths` JSON column on a single row. Add per-user/day caps and a hard byte limit per report (e.g. 50 MB total).

M-7 — `JSON.parse(row.file_paths)` without `try/catch` in `purgeFiles`

`backend/routes/reports.js:99,425,571,597,624`. If the column is corrupted (e.g., manually edited or a partial write), the entire request 500s. Wrap in `try/catch` returning `[]`.

M-8 — Predictable filename generation

`Date.now()-Math.random()*1e9`. Use `crypto.randomUUID()` or `crypto.randomBytes(16).toString('hex')`. Cheap upgrade; also makes H-1 less catastrophic until you add ownership checks.

M-9 — `helmet's CSP style-src 'unsafe-inline' (backend)` and lack of `frame-ancestors`

Backend mostly serves JSON, but `/user-manual` (`backend/server.js:90-92`) and any future server-rendered HTML inherit this. Replace `'unsafe-inline'` with hashed/nonced inline styles, and add `frame-ancestors 'none'`.

M-10 — R2 access creds rely on env config; no scope verification in code

`storage.js` blindly uses `R2_BUCKET` with full `PutObject/GetObject/DeleteObject` grants. There's no path-prefix safety inside the bucket. If `R2_KEY/SECRET` leak (Railway env compromise), the entire bucket is read/write/delete. Use **scoped R2 tokens** limited to the `uploads/` prefix; consider per-environment buckets.

M-11 — `_migrations` table created without a unique constraint check before insert

Re-running migration on race could fail silently. Low likelihood; mention for completeness.

M-12 — Service Worker share-target accepts and caches files unconditionally

`frontend/public/sw.js:29-57` writes any shared blob to cache, no size/type check. Phone could be persuaded to "Share" a giant file → SW cache bloats. Add an `accept` & size guard before `cache.put`. Also: `Response.redirect('/share-pick', 303)` is correct, but ensure `/share-pick` requires auth (it does, via `<PrivateRoute>` in `App.jsx`).

M-13 — Network-first PWA fallback returns synthetic 200 JSON when offline

`frontend/public/sw.js:92-95` returns `{"error": "offline"}` with HTTP 200, swallowing the failure. Frontend code (`api.js` request) checks `res.ok` and treats 200 as success → it will then try `res.json()`, parse the synthetic body, and call `data.error` only if non-200. Not a security vuln but causes silent data integrity issues that hide outages. Use a 503 status on the synthetic response.

M-14 — Export-token endpoints set Cache-Control defaults

No `Cache-Control: no-store` on export responses. Excel "From Web" caches; if a token is rotated, stale state can persist. Set explicitly.

6. Low / Informational

- **L-1** Username regex `/^[A-Za-z ]{1,60}$/` blocks Arabic and most non-Latin names — operational issue at KAIA where staff have Arabic-script names; if you later relax this, beware of injection sinks in the audit log `user` column (currently safe because all writes go through parameterised SQL).
- **L-2** `req.username || req.role` fallback in audit logging (`backend/routes/reports.js` many sites) — once tokens always carry a username (today they do), the `|| req.role` branch is dead code. Remove to clarify.
- **L-3** Helmet's `crossOriginResourcePolicy: 'same-origin'` blocks the file download from being embedded cross-origin (good) but also blocks it from the Vercel-hosted SPA if you're not careful. Verify in production that file downloads still work; if not, set `cross-origin` and rely on the auth check.
- **L-4** `csv_to_flights_json.py` and `csv-to-json.js` are dev-only scripts — confirm they are not deployed (they aren't referenced from `package.json` scripts; OK).
- **L-5** `Procfile` and `railway.json` both define start commands. Harmless duplication, but pick one.

- **L-6** backend/.gitignore not shown — confirm it excludes .env, uploads/, and any local DB files; root .gitignore does (.env, *.db, uploads/).
 - **L-7** Frontend dependency jspdf 4.x — verify you're on ≥4.2.x for the recent ReDoS fix; you are (4.2.1). recharts 2.12.x — current. exceljs 4.4.x — keep an eye on prototype-pollution advisories. Run npm audit in CI.
 - **L-8** No SBOM, no Dependabot/Renovate config in repo. Add dependabot.yml.
 - **L-9** flights.json is loaded with require() at process start — large file rebuilds require restart, and any malformed JSON crashes the worker. Move to fs.readFile with a guarded reload.
 - **L-10** Frontend SharePicker.jsx:63 passes getRole() as user to attachFilesToReport, which sends it as a form field — server doesn't use it. Dead data flow; remove.
 - **L-11** Service worker cache name noshow-v2 — bump on every release or stale UI may be served including stale CSP/token-handling code.
 - **L-12** Bootstrapping the first supervisor requires direct DB insertion (no /bootstrap route). Document this; a hand-rolled SQL with a strong PIN should be on the deployment runbook.
 - **L-13** db.js ssl: { rejectUnauthorized: false } by default (env var defaults to non-'true'). Internal Railway Postgres is fine, but if you ever talk to a customer DB this becomes MITM-able. Default to true.
 - **L-14** seed-test-data.js uses better-sqlite3 and a local noshow.db path that doesn't match the prod Postgres setup. Dev-only; document not to run against prod.
-

7. Quick Wins (≤ 1 hour of work each)

- 1 **Server-side submitted_by/nusuk_by** — set from req.username, ignore body. (C-3)
 - 2 **Equalise login errors and add a dummy bcrypt on the not-found path.** (H-2)
 - 3 **Validate pin.length ≤ 12 server-side in validatePin and on login.** (H-7)
 - 4 **Separate rate limiters for login/register; add limiter to GET /invite/:token.** (H-4, M-1)
 - 5 **Frontend vercel.json headers block** (CSP, HSTS, referrer-policy, nosniff). (H-5)
 - 6 **crypto.timingSafeEqual for the HMAC comparison.** (M-3)
 - 7 **crypto.randomUUID() for upload filenames.** (M-8)
 - 8 **Cache-Control: no-store + Referrer-Policy: no-referrer on /api/export/*.** (C-5)
 - 9 **Reject ?token= on /api/export/*; accept only header.** (C-5)
 - 10 **Enforce export_tokens.role in the two read endpoints.** (C-6)
 - 11 **Hash invite + export tokens at rest (SHA-256).** (M-2)
 - 12 **Add failed_attempts + locked_until to users and per-account logout.** (H-3)
 - 13 **Audit-log login successes/failures, export-token use, deactivations, PIN resets, invite creations, with IP+UA.** (H-9)
-

8. Prioritised Remediation Roadmap

Week 1 — Critical

- C-2: ownership/role checks on report mutation endpoints; forbid non-supervisor `closed` → `under_process`.
- C-3: server-side attribution.
- C-1 (phase 1): add `users.token_version` to HMAC input; bump on PIN reset + deactivate + new "force logout" admin action.
- C-5: header-only export tokens, add `expires_at`, set Cache-Control/Referrer-Policy.
- C-6: enforce export-token roles.

Week 2 — High

- H-1: ownership check on `/api/files/:filename`.
- H-2, H-3, H-4, H-7: harden auth surface.
- H-5: SPA security headers.
- H-9: expand audit log scope.

Month 1 — Medium

- M-2: hash all server-stored tokens.
- M-5: introduce `auditor` (read-only) role.
- M-6: per-user upload quotas.
- M-8: upload filename UUIDs.
- M-10: scope R2 creds.
- H-6: migrate to HttpOnly-cookie sessions + CSRF (deferred; biggest refactor).

Quarter 1 — Strategic

- Implement four-eyes for supervisor sensitive ops (C-4).
 - Externalise logs to an alerting SIEM; build alerts (Section 11).
 - Add chaos test for token revocation flow.
 - Annual third-party pentest + R2-bucket policy review.
-

9. Top Likely Attack Scenarios

- 1 Insider abuse (most likely).** Disgruntled staff edits or reopens another shift's closed reports, deletes attachments before supervisor review, or attributes their actions to a colleague — enabled by C-2 + C-3.
- 2 Shoulder-surfed PIN + device theft.** Operator logs in on a counter terminal, walks away. Bad actor either continues using the active token (12 h window) or, having watched the 6-digit PIN, signs in from elsewhere — enabled by C-1 + weak PIN policy.

- 3 **Lost supervisor phone.** Token persists 24 h despite remote deactivation; finder reads `localStorage`, accesses `/api/users` and `/api/export/tokens`, mints a persistent export token for themselves — C-1 → C-5/C-6.
 - 4 **Phished invite link.** Supervisor pastes invite URL into the wrong channel; an attacker registers first with a chosen name + their PIN — exacerbated by no email tie-in, no notification, no second-factor on registration.
 - 5 **Export token leak.** Excel `From Web` connection saved to OneDrive; the URL contains the token; that OneDrive folder is shared with too many people — full read of reports + audit log via C-5, indefinitely.
 - 6 **Brute-force from botnet.** Distributed PIN guessing across IPs against a known username; no account lockout. Eventually hits a weak PIN — H-3 + H-4.
 - 7 **Stored-PII exfiltration via legit user.** Any staff calls `GET /api/reports` and dumps the full table including nationalities, pax types, comments → portable on USB.
 - 8 **Supply-chain XSS via npm.** A future malicious version of any frontend dep ships JS that reads `localStorage.noshow_token` and beacons it out — H-5, H-6.
-

10. Most Dangerous Vulnerability Chains

Chain A — Insider takeover (Severity: Critical, exploitability: trivial)

C-3 (forge attribution) + C-2 (mutate any report) + H-1 (download any uploaded ID document) + H-9 (no detection) = a staff member can scrape every report, every uploaded passport scan, and rewrite history with another colleague's name on it, undetected.

Chain B — Compromised staff token → supervisor escalation pathway (Severity: High)

H-6 (token in `localStorage`) → C-1 (token cannot be revoked) → H-1 (download all uploads) + C-2 (mutate reports). The staff account can't directly grant supervisor, but can sabotage operations for 24 h after detection.

Chain C — Rogue supervisor lock-out (Severity: Critical)

C-4 (reset peers' PINs / deactivate them) + C-1 (peers' deactivation takes 24 h to bite back) + H-9 (no notification) = a single supervisor can lock every other supervisor out, mint a new supervisor account via invite, exfiltrate everything via export token (C-5/C-6) — well before anyone notices because the only people who would notice are also locked out.

Chain D — Export-token saturation (Severity: High)

C-5 (token in URL, leaks to logs/history) + C-6 (no role enforcement) + H-9 (no usage alerting) + M-14 (no Cache-Control) = a leaked low-privilege link yields full audit log, indefinitely, with no trace.

11. Detection & Monitoring Recommendations

You're currently flying blind. Add, in priority order:

1 Authentication signals (new audit-log actions or external SIEM):

- `login_success` + IP + UA + user
- `login_failure` with same — alert on >5 failures / 5 min per user, or >50 / 5 min global.
- `rate_limited` events.

1 Privilege signals:

- `pin_reset_by` (with both `actor` and `target`).
- `user_deactivated` + `user_activated`.
- `invite_created` with target role.
- `export_token_created` / `rotated` / `revoked` / `deleted` / `used` (with email + ip).

1 Data-access signals:

- Log every `GET /api/reports`, `GET /api/files/:filename` (rate threshold + per-user count baseline).
- Alert on $> \text{baseline} \times 5$ for any single user in an hour.

1 Operational signals:

- Alert on any report transitioning out of `closed`.
- Alert on any `delete` action on reports.

1 Infra:

- Pipe Railway logs + Postgres slow-query log to BetterStack / Datadog / Loki.
- R2 access log → bucket usage anomaly alerts.

1 Synthetic monitoring:

- A canary export token used only by a healthcheck; alert if its `last_used` is updated by anything other than the healthcheck source IP.

12. Incident-Response Readiness Gaps

- **No way to force-logout a single user**, let alone everyone (fix part of C-1).
- **No way to rotate the SESSION_SECRET cleanly** — rotating it invalidates every active token at once, no overlap window. Should run dual-secret support during rotation.
- **No backup-restore drill documented**, no RPO/RTO commitments visible in repo.
- **No alerting destination** (no Slack/Teams/email integration for security events).
- **No documented playbook** for "supervisor account compromise," "stolen device," "exposed export token," "leaked invite link," "DB credential leak," "R2 credential leak."
- **No legal-hold mechanism** — closed reports auto-delete attachments after status change and after auto-close. If you need to retain evidence for a complaint/dispute, attachments are already gone. Add a "frozen" status that prevents purge.

- **Audit log itself is mutable from Postgres**, no append-only constraint, no off-box ship. Add a daily export to immutable storage (e.g., R2 with object-lock).

13. Secure Architecture Improvements

- **Sessions:** Move to HttpOnly secure cookies + per-user `token_version`. The HMAC scheme is fine if you add (a) `iat/exp/version` into the signed payload and (b) a server-side per-user secret bump for revocation.
- **Identity:** Consider SSO/Microsoft Entra ID for KAIA staff. PINs on shared devices are operationally simple but a security ceiling.
- **Authorization:** Introduce explicit RBAC table (`role`, `permission`) and a small policy module — today `requireRole('supervisor')` is hard-coded in 20+ places; one missed call leaks privileged ops.
- **Resource ownership:** Reports own an `owner_user_id`; default policy is `owner OR supervisor`. Apply uniformly.
- **Object storage:** Per-report prefixes (`uploads/<report_id>/<uuid>.<ext>`) plus signed-URL access with 60 s TTL; never proxy bytes through the API tier.
- **Database:** Hash all token-shaped secrets (SHA-256, with `crypto.timingSafeEqual` on lookup). Add `users.last_login_at`, `users.last_login_ip`, `users.token_version`, `users.failed_attempts`, `users.locked_until`. Add `created_by_user_id` FKs on `reports`, `invite_tokens`, `export_tokens`.
- **Frontend:** SPA CSP. Drop `localStorage` for auth. Re-issue Service Worker on every deploy (version stamp).
- **Backend:** Add `helmet({ hsts: { maxAge: 63072000, preload: true } })`. Add structured logging (Pino) with correlation IDs.
- **Operations:** Dependabot + npm-audit-gate in CI. Periodic `bcrypt` cost re-tuning. R2 scoped keys per env.

14. Zero-Trust Posture Recommendations

- 1 **Never trust the client for identity or authorization** — fix C-3 first, it's the cleanest signal of the bad pattern.
- 2 **Every request re-validates user state, not just token signature** — fix C-1.
- 3 **Least privilege everywhere** — fix C-2 (per-report), C-6 (export roles), M-5 (read-only role).
- 4 **Short-lived everything** — session token 1 h, export tokens 90 d max, invites 6–24 h (already), download URLs 60 s.
- 5 **Encrypt in transit & at rest** — TLS already; ensure Postgres encryption at rest is enabled (Railway managed), R2 SSE-S3 default on.
- 6 **Audit and alert on every privileged operation** — fix H-9.
- 7 **Segregate duties** — no single supervisor can act alone on peer-affecting actions (fix C-4).
- 8 **Make compromise survivable** — daily off-box backups, immutable audit-log copies, documented break-glass account, drill it.

15. Security Maturity Assessment

Domain	Maturity	Evidence
Identity & Access	2 / 5 Repeatable	PIN+name w/ bcrypt; HMAC tokens; no revocation; no MFA; no SSO; no 4-eyes
Application Security	3 / 5 Defined	Helmet, CORS, parameterised SQL, no XSS sinks; IDOR class is broad
Data Protection	2 / 5 Repeatable	TLS, bcrypt, R2 encryption likely on; PII held without segregation; no DLP
Logging & Monitoring	1 / 5 Initial	DB audit_log for some actions; no log shipping, no alerting
Vulnerability Management	1 / 5 Initial	No SBOM, no Dependabot, no scheduled scans
Incident Response	1 / 5 Initial	No playbooks, no detection, no revocation primitives
Secure SDLC / Supply Chain	1 / 5 Initial	No CI security gates, no pinned image, no SBOM, no signed artefacts
Cloud Posture	2 / 5 Repeatable	Managed services, env-based config; bucket scope and IAM unverified
Business Continuity	2 / 5 Repeatable	Auto-restart on Railway; backup/restore drill not in repo
Governance	1 / 5 Initial	No policies in repo; PDPL/data-classification not addressed

Overall: ~1.6 / 5 (Initial-to-Repeatable). Realistic target after the Week-1/Week-2 plan: **2.5–3**.

16. What I'd Want Approval to Test Next

Per the engagement rules, no active probing was performed. With approval (each is non-destructive but interacts with the live system; explicit go-ahead needed for each):

- 1 Verify the proxy/spoofed-XFF rate-limiter bypass against `/api/auth/login` (single request with a forged `X-Forwarded-For` and an obviously-invalid PIN — confirms whether the limiter is sound). Safe / one packet.
- 2 Demonstrate C-2 IDOR by creating a benign test report under one staff account and `PATCH`ing it from another. Safe / two requests / test data deleted afterward.
- 3 Verify the export-token role gap (C-6) by creating a `view`-role token and fetching `/api/export/audit-log`. Safe / one request.
- 4 Verify token-revocation gap (C-1) by capturing a valid token, deactivating the account from a supervisor session, and confirming the captured token still responds. Safe / observation only.
- 5 Verify file-IDOR (H-1) by uploading a marker file under report A as user X, then downloading it as user Y. Safe.
- 6 Inspect deployed response headers on Railway and Vercel (`curl -I`) to confirm H-5 and M-9 in production. Safe.

Additional information requested

- Whether the Railway backend is reachable directly (bypassing Cloudflare/Railway edge proxies) — affects H-4 severity.
- Whether R2 bucket has a public ACL or a public bucket policy (affects H-1 and M-10 severity dramatically).
- Whether DB backups exist, where they live, and who can read them (affects M-2 severity).
- Whether there is a documented bootstrap supervisor, MFA expectations, or external SOC integration.